

UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 3: ENGINEERING TEXT-TO-IMAGE GENERATOR

Question	Generate an engineering-related image, such as a bridge or robotic system, from a suitable text prompt using a pre-trained text-to-image model.
Folder Name	Q3_Engineering_Text_to_Image
Python File	text_to_image.py
Application	Streamlit Web Application
AI Service	Hugging Face Inference API with a pre-trained text-to-image model

1. AIM

To develop a Streamlit-based AI application that accepts an engineering-related text prompt and generates a corresponding image using a pre-trained text-to-image generative AI model.

2. OBJECTIVE

- Convert a natural-language engineering description into an image.
- Use a pre-trained text-to-image model instead of training a model from scratch.
- Provide a simple web interface for entering prompts and viewing generated images.
- Demonstrate the application of multimodal generative AI in engineering.

3. PROBLEM STATEMENT

Engineering concepts are often easier to understand when they are represented visually. The objective of this implementation is to create an AI system that can transform an engineering description, such as a suspension bridge or robotic arm, into a visual representation automatically.

4. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose	Role in the Application
Python	Application development	Implements the complete application
Streamlit	Web interface	Provides prompt input and image display
Hugging Face	AI model inference	Provides access to the pre-trained text-to-image model
InferenceClient	Model API communication	Sends the prompt and receives the generated image
Pillow (PIL)	Image handling	Supports displaying the returned image

5. SYSTEM WORKFLOW

Workflow of the Engineering Text-to-Image System

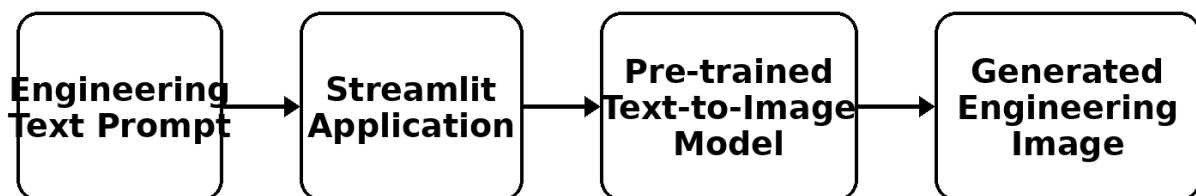


Figure 1. Workflow of the Engineering Text-to-Image Generation System

The user first enters an engineering-related text prompt. The Streamlit application sends this prompt to a pre-trained text-to-image model through the Hugging Face inference service. The model processes the text and generates a visual image, which is returned to the application and displayed to the user.

6. STEP-BY-STEP IMPLEMENTATION

Step 1 – Create the project folder

Create the following folder inside the UNIT_IV_Multimodal_Generative_AI project:

Q3_Engineering_Text_to_Image

Step 2 – Create the Python file

Inside the folder, create:

text_to_image.py

Step 3 – Install required libraries

Install Streamlit, Hugging Face Hub and Pillow in the active Python environment:

```
pip install --upgrade huggingface_hub streamlit pillow
```

Step 4 – Create a Hugging Face access token

Create an access token in the Hugging Face account settings and store it as the HF_TOKEN environment variable. The token should not be written directly into the Python source code.

Step 5 – Import the required modules

The application uses Streamlit for the interface, InferenceClient for communication with the Hugging Face model service, and os for reading the API token from the environment.

Step 6 – Create the model client

An InferenceClient object is initialized with the Hugging Face token. The client is then used to call a pre-trained text-to-image model.

Step 7 – Accept the engineering prompt

A Streamlit text area is provided so that the user can enter a detailed engineering description.

Step 8 – Generate the image

When the user clicks the Generate Engineering Image button, the prompt is sent to the text-to-image model. The returned image is displayed in the Streamlit application.

Step 9 – Run the application

Run the application from the project root using:

```
streamlit run Q3_Engineering_Text_to_Image\text_to_image.py
```

7. SAMPLE PROMPT USED

A modern suspension bridge with steel cables, realistic engineering design, strong structural support, daylight, highly detailed.

8. IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
import os
```

```
st.set_page_config(
    page_title="Engineering Text-to-Image Generator",
    page_icon="🏗️",
    layout="wide"
)

# Initialize InferenceClient
token = os.getenv("HF_TOKEN")
client = InferenceClient(token=token)
```

UNIT IV – Multimodal Generative AI | Q3 Engineering Text-to-Image

```

    page_icon="🏗️",
    layout="wide"
)

st.title("🏗️ Engineering Text-to-Image Generator")

st.write(
    "Generate engineering-related images from suitable text prompts "
    "using a pre-trained text-to-image model."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error("HF_TOKEN is not configured.")
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

prompt = st.text_area(
    "Text Prompt:",
    value="A modern suspension bridge with steel cables, "
        "realistic engineering design, strong structural support, "
        "daylight, highly detailed",
    height=120
)

if st.button("🔄 Generate Engineering Image"):
    if not prompt.strip():
        st.warning("Please enter an engineering-related prompt.")
    else:
        with st.spinner("Generating image using AI..."):
            image = client.text_to_image(
                prompt=prompt,
                model="black-forest-labs/FLUX.1-schnell"
            )

        st.success("Image generated successfully!")
        st.image(
            image,
            caption="AI-Generated Engineering Image",
            width="stretch"

```

)

9. OUTPUT SCREENSHOTS

Engineering Text-to-Image Generator

Generate engineering-related images from suitable text prompts using a pre-trained text-to-image model.

Enter Engineering Image Prompt

Text Prompt:

A modern suspension bridge with steel cables, realistic engineering design, strong structural support, daylight, highly detailed

 Generate Engineering Image

Image generated successfully!

Figure 2. Streamlit interface showing the engineering image prompt and successful generation message.

Generated Engineering Image

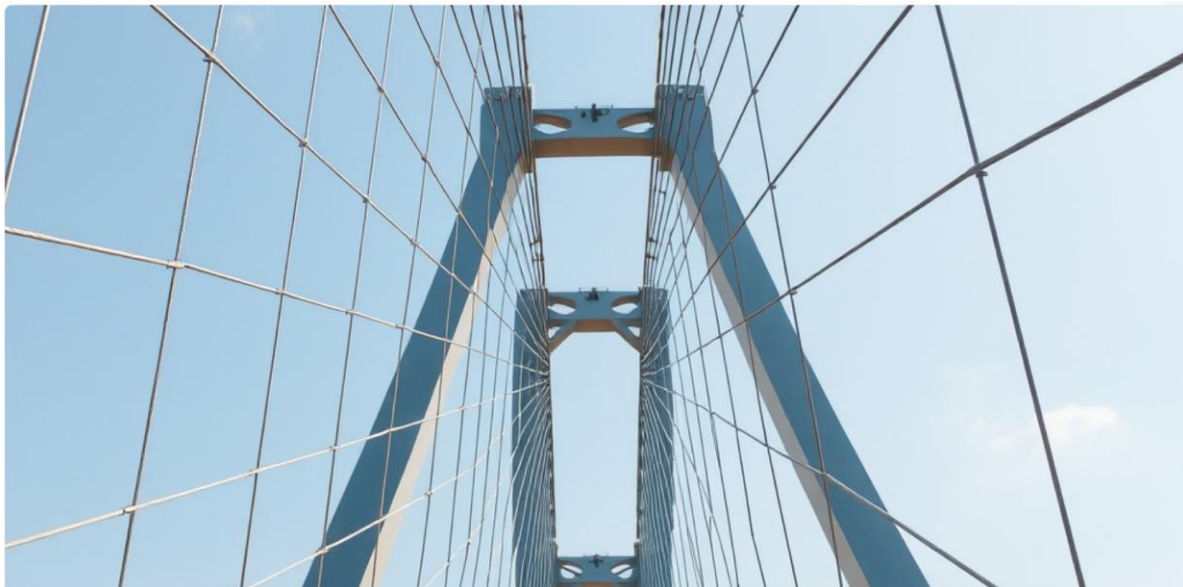


Figure 3. Generated engineering image – suspension bridge.

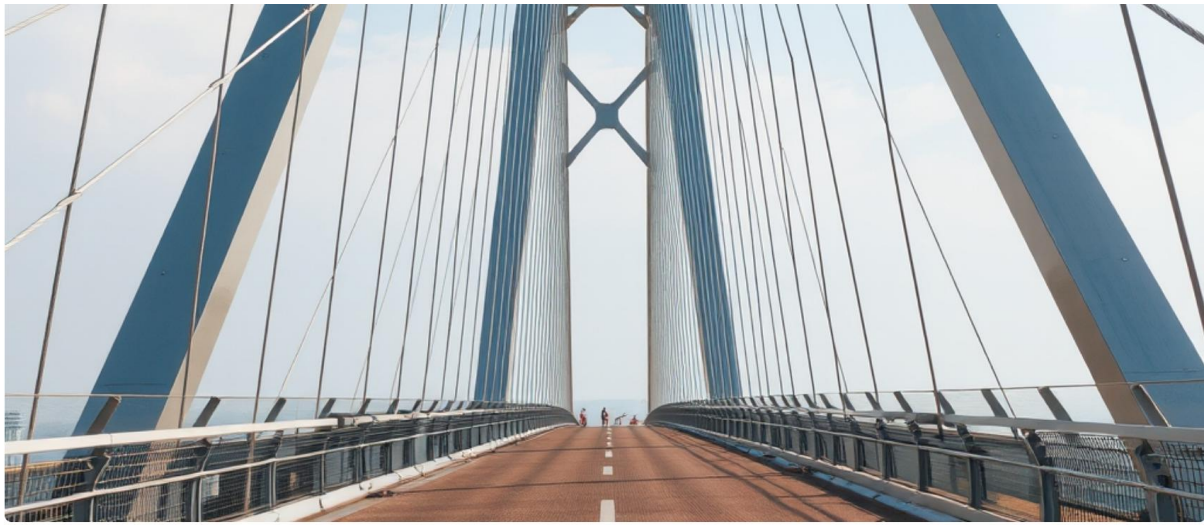


Figure 4. Generated image output showing the detailed bridge structure.

10. RESULT

The Engineering Text-to-Image Generator was successfully implemented. The application accepts an engineering-related text prompt and generates a corresponding engineering image using a pre-trained text-to-image model. The demonstrated prompt produced a detailed suspension-bridge image, confirming that the system works successfully.

11. APPLICATIONS

- Visualization of civil engineering structures such as bridges and buildings.
- Concept visualization for mechanical engineering designs.
- Early-stage visualization of robotic systems and machines.
- Educational demonstrations and engineering presentations.
- Rapid generation of visual concepts from natural-language descriptions.

12. ADVANTAGES

- Easy-to-use web interface.
- No model training is required.
- Converts text descriptions into visual content.
- Useful for rapid engineering concept visualization.
- Can generate different images by changing the prompt.

13. CONCLUSION

The Engineering Text-to-Image Generator demonstrates how multimodal generative AI can convert natural-language engineering descriptions into visual representations. By combining Streamlit with a pre-trained text-to-image model available through Hugging Face inference, the system provides a practical and user-friendly approach for engineering visualization. The successful generation of the suspension-bridge image confirms that the required implementation has been achieved.